

ConceptGraph AI - 跨学科知识图谱智能体

项目概述

项目名称: 《智能体云原生开发》期末大作业命题三: 跨学科知识图谱智能体

GitHub仓库: https://github.com/kaifenger/Final_Cloud_Computing

协作者: 10235501435张凯诚(kaifenger)、10235501452肖璟仪(ezzzb7)

(详细分工见附录)

成员	GitHub账号	主要职责	贡献比例
肖璟仪	ezzzb7	算法/Agent组 - LLM Agent设计、Prompt工程、多源验证、相似度计算	50%
张凯诚	kaifenger	架构/工程组 - Docker部署、FastAPI后端、React前端、Neo4j/Redis集成	50%

一、项目背景与痛点分析

1.1 痛点场景

现代知识学习面临严重的**知识碎片化**问题。以"神经网络"为例,学习者在计算机科学课程中接触该概念时,往往只获得深度学习领域的应用知识,却难以理解其与生物神经科学、高等数学、信息论的深层联系。

核心痛点包括:

- 概念关联不可见:** 学习者不知道"神经网络"源自生物神经元结构,不清楚其数学本质是梯度下降优化,无法联想其与物理学能量最小化原理的结构同构性。
- 远亲概念难发现:** 传统搜索引擎只能找到"近亲概念"(神经网络→卷积神经网络),无法挖掘"远亲概念"(神经网络→霍普菲尔德网络→统计物理自旋玻璃模型)。这些原理相通的跨学科概念是创新思维的重要源泉,但现有工具难以系统性发现。
- 知识网络无可视化:** 即使通过文献综述梳理出跨学科关联,学习者也缺乏直观工具将概念关系以图谱形式呈现,导致记忆困难、理解不深。

1.2 核心目标

本项目设计了基于LLM智能体的跨学科知识图谱系统,实现三大核心目标:

- 自动化跨学科关联挖掘:** 用户输入核心概念(如"熵"),系统通过LLM智能体强制在数学、物理、化学、生物、计算机、社会学六大学科搜索,自动发现远亲概念。例如输入"熵",系统挖掘出信息论的香农熵、热力学的玻尔兹曼熵、机器学习的交叉熵损失函数等。
- 结构化图谱构建:** 系统将概念及关系以标准JSON数据结构存储,包含节点(概念名称、所属学科、权威定义、可信度评分)和边(关联类型、关联强度、关联原因),确保数据可复用、可

扩展。

3.交互式知识网络可视化：基于D3.js力导向图算法，将抽象概念关系转化为可交互图谱。用户可点击节点查看详情、展开子图、进行AI问答，实现"知识探索-深度理解-网络扩展"的完整学习闭环。

二、系统架构设计

2.1 技术栈选型

云原生组件：

- **Docker:** 容器化部署,实现开发/生产环境一致性
- **Neo4j:** 图数据库,持久化存储知识图谱
- **Redis:** 分布式缓存,1小时TTL提升响应速度
- **Docker Compose:** 多服务编排,一键启动

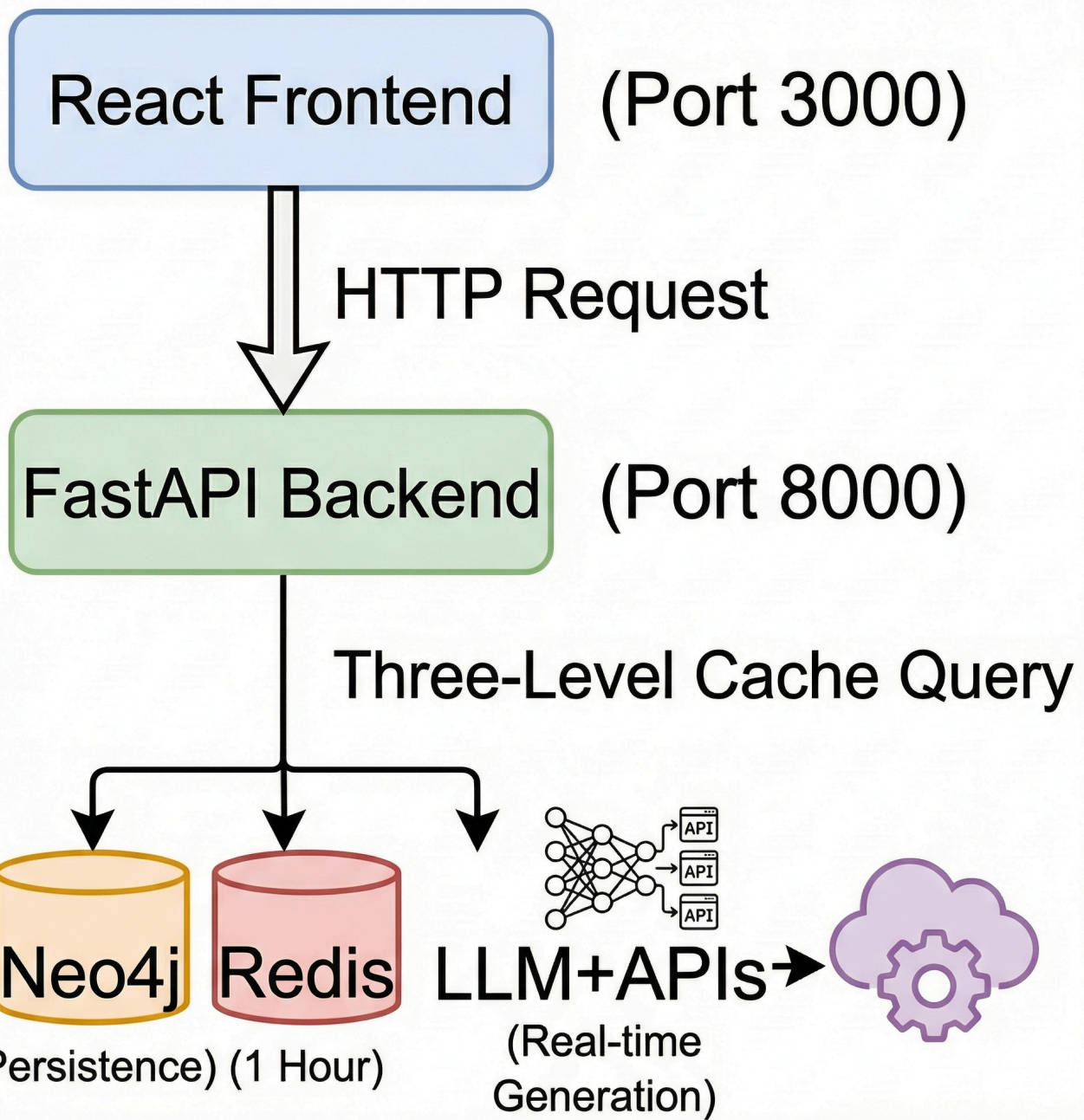
后端技术：

- **Python 3.11 + FastAPI:** 高性能异步Web框架
- **OpenRouter + Google Gemini Flash:** LLM推理引擎
- **OpenAI Embeddings:** 语义相似度计算(text-embedding-3-small)
- **Wikipedia API + Arxiv API:** 多源知识验证

前端技术：

- **React 18 + TypeScript 5:** 类型安全的前端框架
- **D3.js 7:** 力导向图可视化库
- **Ant Design 5:** UI组件库
- **Axios:** HTTP客户端

2.2 系统架构图



系统采用微服务架构，分为四层：

展示层 (React Frontend)：运行在端口3000，提供交互式用户界面。React 18+TypeScript 5确保类型安全，D3.js 7用于图谱渲染，Ant Design 5提供统一UI组件。前端通过Axios发送HTTP请求至后端API，实现前后端完全解耦。

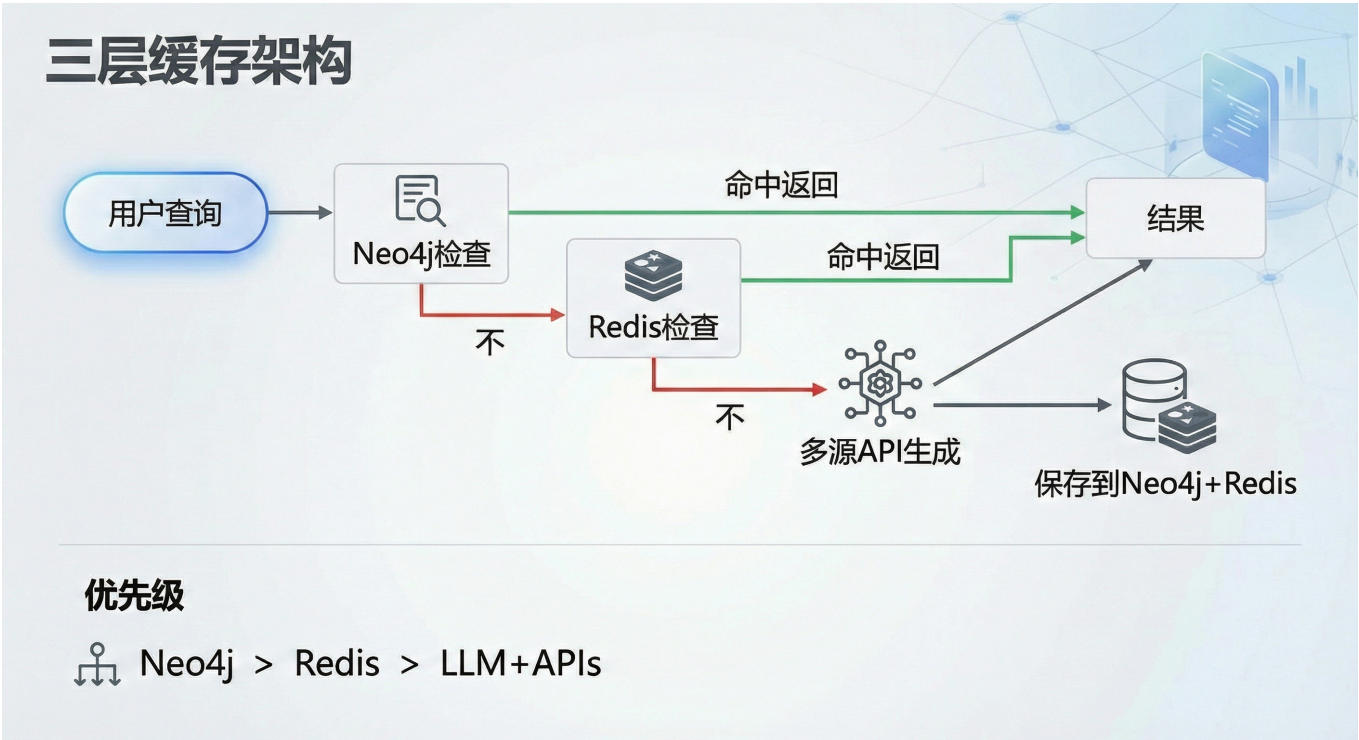
应用层 (FastAPI Backend)：运行在端口8000，是业务逻辑核心处理层。FastAPI框架的异步特性使其能够并行调用多个外部API（OpenRouter、Wikipedia、Arxiv），显著降低响应时间。后端设计了6个核心API接口，包括概念发现 (/discover)、节点展开 (/expand)、AI问答 (/ai/chat) 等，所有接口遵循RESTful规范并自动生成OpenAPI文档。

数据层 (三级缓存策略)：第一级为Neo4j图数据库（端口7687），提供永久持久化存储，支持复杂图查询（最短路径、子图提取）；第二级为Redis缓存（端口6379），存储1小时TTL的热点数据，命中率约40%，响应速度提升10-100倍；第三级为实时LLM生成，当缓存未命中时才调用，确保任意概念可查询。

智能体层 (Agent System)：系统的AI大脑，采用三层编排架构。ConceptDiscoveryAgent负责跨学科概念挖掘，VerificationAgent负责多源验证和可信度评分，GraphBuilderAgent负责将概念关系转化为图结构。三个Agent通过AgentOrchestrator统一调度，实现流水线式处理。

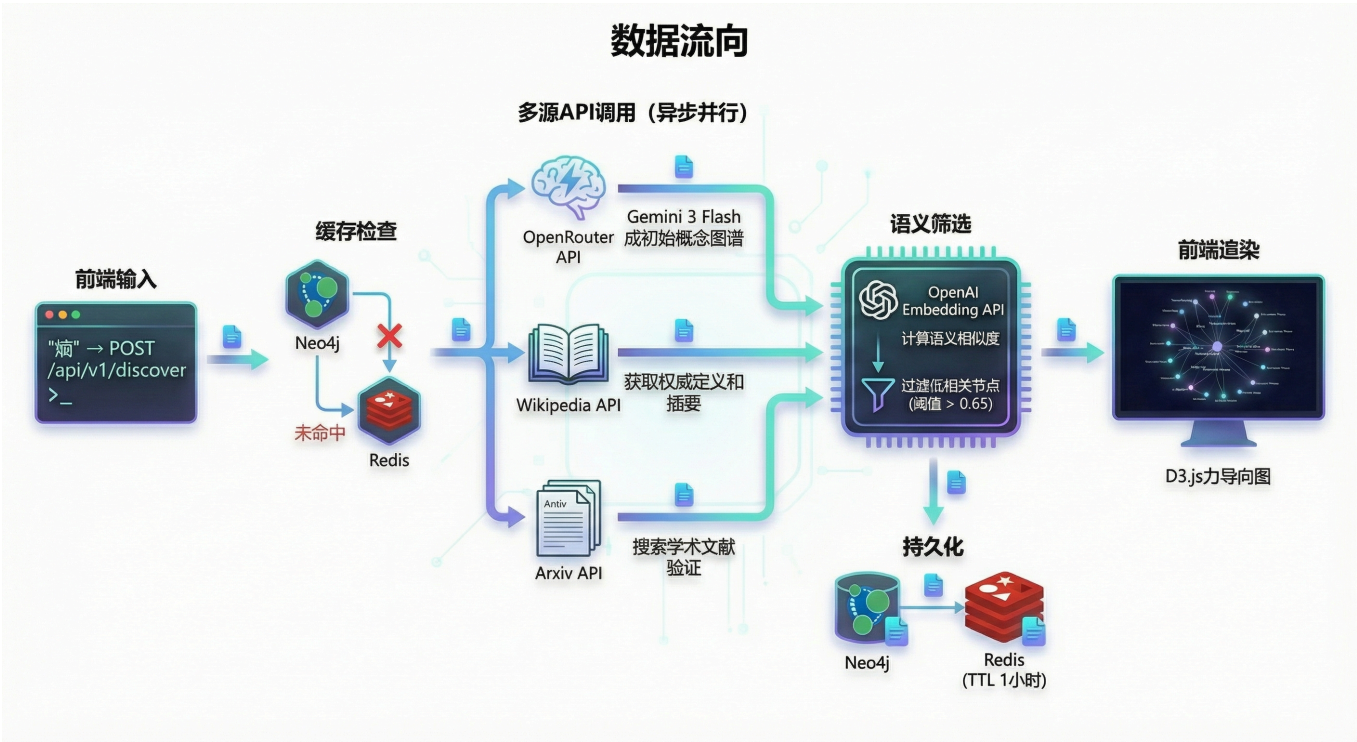
2.3 三层缓存架构

项目实现的三层缓存策略提升系统响应速度：



Neo4j永久存储,支持复杂图查询； **Redis**: 热数据缓存,避免重复LLM调用； **LLM**: 兜底生成,确保任意概念可查询

2.4 数据流向图



完整流程:

1. 前端输入 → POST /api/v1/discover
2. 缓存检查 → Neo4j → Redis → (未命中)
3. 多源API调用(异步并行):
 - Gemini 3 Flash → 初始概念生成
 - OpenRouter API → 成熟概念图谱
 - Wikipedia API → 权威定义和描述
 - Arxiv API → 学术文献验证
4. 语义筛选 → OpenAI Embedding API → 计算相似度(阈值0.65)
5. 持久化 → 保存到Neo4j + Redis(TTL 1小时)
6. 前端渲染 → D3.js力导向图

三、核心功能实现

3.1 功能清单

功能	描述	API端点
功能1: 自动跨学科发现	输入单一概念,系统自动挖掘6大学科关联	POST /api/v1/discover
功能2: 指定学科发现	输入概念+指定学科,定向挖掘	POST /api/v1/discover/disciplined
功能3: 多概念桥梁发现	输入多个概念,寻找跨学科桥梁	POST /api/v1/discover/bridge

功能	描述	API端点
节点展开	点击节点扩展子图	POST /api/v1/expand
AI问答	基于概念背景的智能问答	POST /api/v1/ai/chat
学术论文检索	查询Arxiv相关论文	GET /api/v1/arxiv/search

3.2 LLM Agent工具链

3.2.1 Agent架构(三层编排)



3.2.2 核心Prompt设计

功能1: 跨学科概念生成Prompt (位置: backend/api/real_node_generator.py)
设计原则：强制LLM从六大学科领域搜索，每个领域至少生成1个概念，打破学科壁垒。
关键指令：

- 明确"远亲概念"判定标准：数学形式相同但应用领域不同、底层原理一致、历史启发关系、结构同构性
- 输出格式： 概念名 | 学科 | 关系类型 | 跨学科原理
- 参数配置：temperature=0.4 （平衡创造性与准确性），max_tokens=800，reasoning=enabled （Gemini推理模式）

```
System： 你是跨学科知识挖掘专家,擅长发现不同领域间的深层原理关联和结构同构性。

User Prompt：
"""
你是一个跨学科知识挖掘专家。请为概念"{parent_concept}"挖掘{max_count}个跨领域的相关概念。

【核心任务】发现跨学科的"远亲概念" - 不同领域中原理相通的概念
```

【跨学科搜索策略】

必须从以下不同领域寻找概念（每个领域至少1个）：

- 1. 数学/统计学：寻找数学本质、理论基础
- 2. 物理学：寻找物理类比、能量/信息原理
- 3. 生物学/神经科学：寻找仿生学启发、演化机制
- 4. 计算机科学：寻找算法实现、工程应用
- 5. 社会学/经济学：寻找群体行为、博弈模型
- 6. 其他交叉学科：心理学、语言学、复杂系统等

【远亲概念判定标准】

好的远亲概念（必须满足至少一条）：

- 数学形式相同但应用领域不同（如：熵在热力学vs信息论）
- 底层原理一致（如：神经网络vs生物神经元）
- 历史启发关系（如：遗传算法vs达尔文进化论）
- 结构同构（如：社交网络vs蛋白质网络）

【输出格式】（每行一个概念）

概念名 | 学科 | 关系类型 | 跨学科原理

示例：

神经网络 | 生物学 | bio_inspired | 模拟生物神经元的突触连接和激活传播机制
信息熵 | 物理学 | mathematical_analogy | 与热力学熵数学形式完全一致($H = -\sum p \cdot \log(p)$)
"" ""

功能2: 指定学科挖掘Prompt

针对性学习场景，用户明确研究方向后仅在特定学科搜索。

关键约束：

- 严格要求"只能从指定学科中选择概念"
- 每个学科至少生成1个概念（若可能）
- 必须解释概念在该学科视角下的关联

"" ""

任务：为概念"{parent_concept}"在指定学科中生成{max_count}个相关概念。

****仅在以下学科中寻找**：**

{discipline_list}

严格要求：

- 1. ****只能从上述指定学科中选择概念****
- 2. 每个学科至少生成1个概念（如果可能）
- 3. 必须解释每个概念与"{parent_concept}"在该学科视角下的关联

输出格式：

概念名 | 学科 | 关系类型 | 学科视角下的关联原理
"" ""

功能3: 多概念桥梁发现Prompt

创新性模式，寻找能同时连接多个概念的"桥梁概念"。
桥梁概念定义：一个概念能同时与所有输入概念产生跨学科关联，揭示概念背后的共同原理。

"" ""
任务：找到能连接以下所有概念的"桥梁概念"。

输入概念: {concepts}

【桥梁概念定义】
- 一个概念,能同时与所有输入概念产生跨学科关联
- 必须揭示这些概念背后的共同原理

【输出格式】
桥梁概念 | 学科 | 与概念A的关联 | 与概念B的关联 | ...
"" ""

3.2.3 多源验证与可信度评分

可信度计算公式：

credibility = base_credibility × (0.7 + 0.3 × similarity)

其中：
- base_credibility = 0.95 (Wikipedia验证) / 0.70 (仅LLM生成)
- similarity = cosine_similarity(concept1_embedding, concept2_embedding)

多源验证逻辑：

- Wikipedia权重0.7（权威来源）
- Arxiv权重0.9（学术文献）
- LLM权重0.3（降低幻觉风险）

3.3 语义相似度计算

技术方案：OpenAI Embeddings + 余弦相似度
计算流程：

1. 使用OpenAI text-embedding-3-small模型将概念转换为1536维向量
2. 计算余弦相似度： cosine_sim = (v1·v2)/(||v1||×||v2||)

3. 归一化到[0,1]范围

动态阈值筛选：设置阈值0.65，LLM生成20个候选后，按相似度排序动态筛选，最终保持3-9个高质量节点。

3.4 前端可视化实现

前端首页搜索界面 - 展示三种功能模式选择



D3.js力导向图配置：

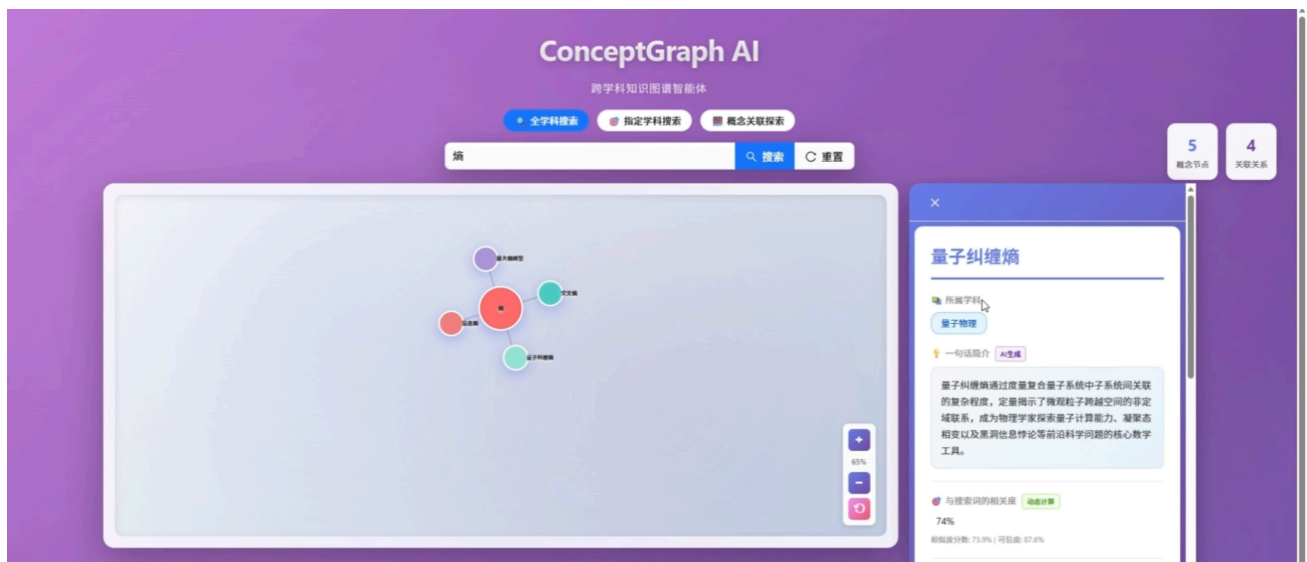
核心参数设计：

- **链接力 (link force)：**根据节点深度调整距离，根节点到子节点140像素，子节点间100像素，确保层次清晰
- **电荷力 (charge force)：**设置为-400，保证节点互斥避免重叠
- **碰撞力 (collision force)：**半径40像素，防止节点文字重叠
- **桥梁模式：**输入节点到桥接节点距离拉长至180像素，形成三角形布局

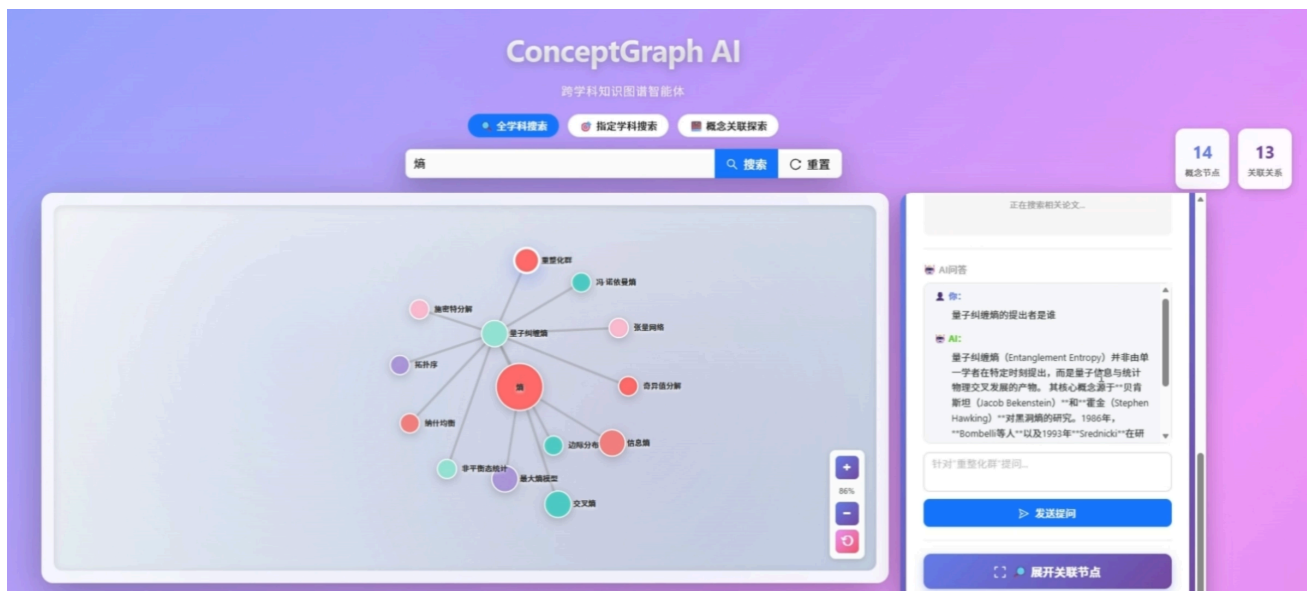
节点交互功能：

1. **节点点击：**展示8部分详情（概念名、学科、定义、简介、可信度、相似度、来源、Wiki链接）
2. **节点展开：**以当前节点为父节点，生成2倍数量候选，经相似度筛选后添加到图谱
3. **节点拖拽：**支持手动调整节点位置
4. **缩放/平移：**支持50%-300%缩放，全方位平移
5. **AI问答：**基于节点背景的智能问答
6. **学术论文：**自动检索Arxiv相关论文Top5

功能1结果展示 - "神经网络"的跨学科知识图谱， 点击节点后有信息展示



节点展开动画 - 展开某节点后新增子节点的效果

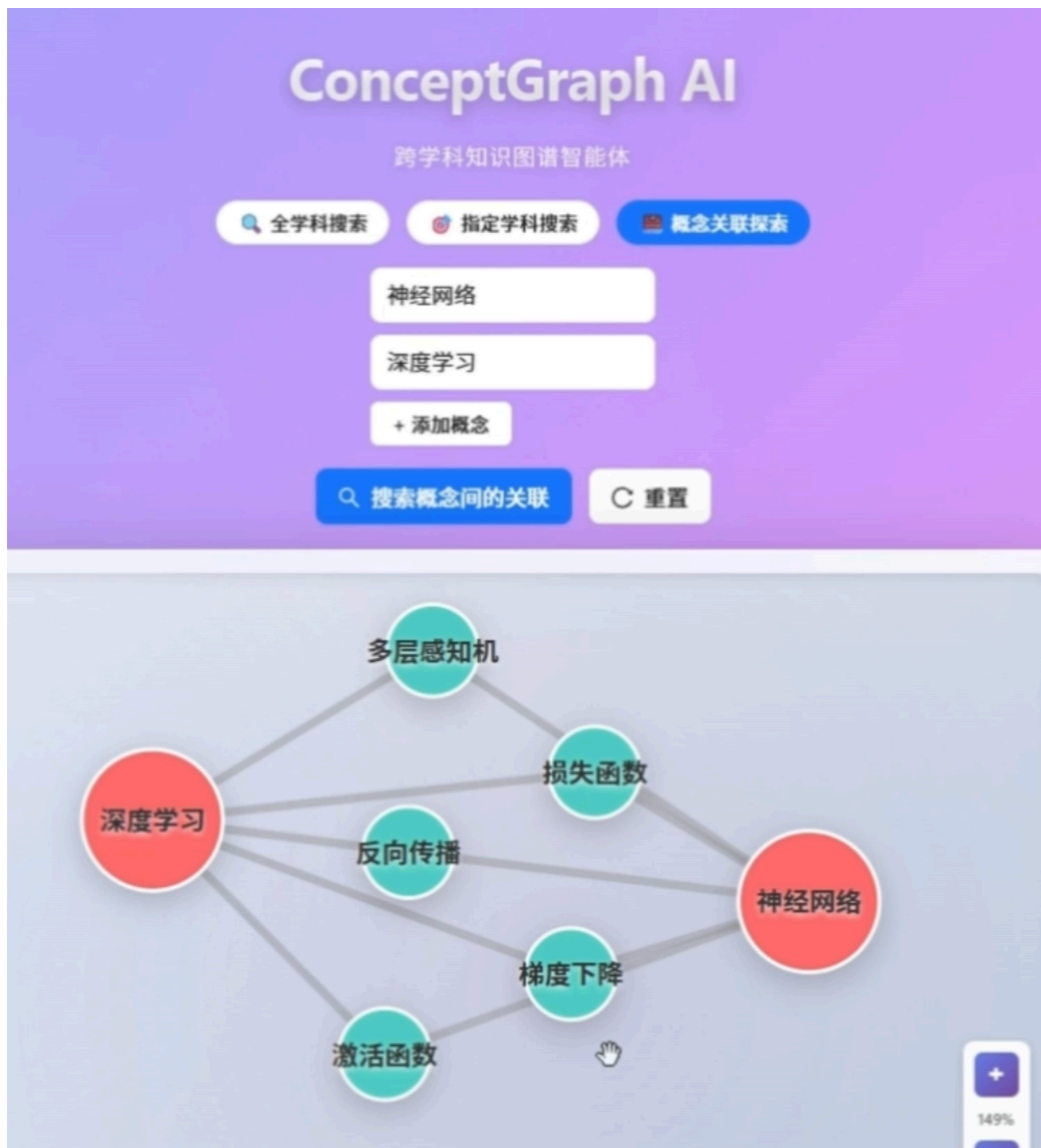


AI问答界面 - 在节点详情中进行智能问答的交互



一学者在特定时刻提出，而是量子信息与统计物理交叉发展的产物。其核心概念源于**贝肯斯坦（Jacob Bekenstein）**和**霍金（Stephen Hawking）**对黑洞熵的研究。1986年，**Bombelli等人**以及1993年**Srednicki**在研究黑洞熵的几何起源时，明确提出了通过对子系统求偏迹来定义几何熵（即纠缠熵）的方法。此外，**Bennett等人**在90年代对量子纠缠纯度的度量也做出了奠基性贡献。

多概念桥梁模式 - 功能3的桥梁概念可视化效果



四、Docker部署方案

4.1 Docker Compose配置

Docker部署架构图：

```
PS D:\大三\云计算\云计算作业\期末> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
12081fd42cbc	conceptgraph-frontend	"docker-entrypoint.s..."	6 minutes ago	Up 6 minutes	0.0.0.0:3000->3000/tcp
e075ba3d7b74	conceptgraph-backend	"unicorn main:app --..."	6 minutes ago	Up 6 minutes	0.0.0.0:8000->8000/tcp
c603c561eb9b	redis:7.2-alpine	"docker-entrypoint.s..."	6 minutes ago	Up 6 minutes	0.0.0.0:6379->6379/tcp
5eaebef54da6	neo4j:5.15.0	"tini -g -- /startup..."	6 minutes ago	Up 6 minutes	0.0.0.0:7474->7474/tcp
6fecc28db042	registry	"/entrypoint.sh /etc..."	5 weeks ago	Up 35 minutes	0.0.0.0:5000->5000/tcp

```
PS D:\大三\云计算\云计算作业\期末>
```

系统通过Docker Compose实现一键部署，包含4个服务：

Neo4j服务：

- 镜像：neo4j:5.15.0
- 端口：7474（HTTP浏览器）、7687（Bolt协议）
- 配置：2G堆内存，APOC插件支持
- 数据持久化：neo4j_data、neo4j_logs、neo4j_plugins

Redis服务：

- 镜像：redis:7.2-alpine
- 端口：6379
- 配置：AOF持久化，密码认证
- 数据持久化：redis_data

Backend服务：

- 构建：基于backend/Dockerfile
- 端口：8000
- 依赖：neo4j、redis
- 环境变量：数据库连接信息、API密钥

Frontend服务：

- 构建：基于frontend/Dockerfile
- 端口：3000
- 依赖：backend
- 环境变量：后端API地址

4.2 一键启动脚本

提供跨平台启动脚本：

Windows (start.bat)：

```
docker-compose up --build -d
echo [SUCCESS] 服务启动成功！
echo - Frontend: http://localhost:3000
```

```
echo - Backend API: http://localhost:8000/docs
echo - Neo4j Browser: http://localhost:7474
```

Linux/Mac (start.sh):

```
docker-compose up --build -d
echo "[SUCCESS] 服务启动成功!"
```

五、云原生技术体现

5.1 容器化部署 (Docker)

所有服务容器化:

- Backend: Python 3.11-slim基础镜像 (120MB)
- Frontend: Node 18-alpine基础镜像 (90MB)
- Neo4j: 官方neo4j:5.15.0镜像
- Redis: 官方redis:7.2-alpine镜像

多阶段构建优化:

- 分离依赖安装和代码拷贝步骤
- 充分利用Docker缓存层机制
- 最小化最终镜像体积

5.2 服务编排 (Docker Compose)

网络隔离: 自定义bridge网络conceptgraph-network, 服务间通过服务名通信

数据持久化: 4个volume确保容器重启后数据不丢失

依赖管理: depends_on指令确保启动顺序 (Neo4j/Redis → Backend → Frontend)

健康检查: restart: unless-stopped自动重启策略

5.3 微服务架构

```
用户请求 → Nginx(可选) → Frontend(3000) → Backend(8000)
                                     ↓
                               Neo4j(7687)
                               Redis(6379)
                               LLM APIs(外部)
```

关键特性:

- **服务解耦:** 前后端分离, 数据库独立部署
- **水平扩展:** 支持多副本部署 (docker-compose scale)

- 故障隔离：单个服务故障不影响其他服务

5.4 云原生设计原则

原则	实现
12-Factor App	环境变量配置(. env文件)、无状态服务、日志输出到stdout
容错性	Redis缓存降级、数据库重连机制、LLM超时保护
可观测性	健康检查接口(/health, /ready)、详细日志记录
API优先	RESTful API设计、OpenAPI文档(/docs)

六、稳定性与扩展性设计

6.1 容错机制

6.1.1 LLM幻觉处理

三重防护机制：

第一重：学术概念过滤器

- 使用LLM二元分类器过滤非学术内容
- Prompt: "你是学术概念过滤器。'{概念}' 是学术概念吗？只回答'是'或'否'。"
- 参数：temperature=0.1确保输出稳定
- 效果：拦截如"笨蛋"等非学术输入

第二重：多源交叉验证

- Wikipedia存在条目：可信度基础分0.95
- 仅LLM生成：可信度基础分0.70
- Arxiv检索到论文：额外加权0.1
- 权威来源的概念获得优先展示

第三重：可信度阈值过滤

- 最低可信度阈值0.5
- 约30%的LLM生成概念因可信度不足被过滤
- 有效降低幻觉风险

6.1.2 网络超时保护

分层超时策略：

- Wikipedia API：10秒（响应通常1-2秒）
- Arxiv API：5秒（用户耐受上限）

- OpenRouter LLM：90秒（推理模式需要更长时间）
超时处理：
- Wikipedia超时：跳过验证，使用LLM定义但降权
- Arxiv超时：论文列表返回空数组，不影响核心功能
- LLM超时：返回错误提示，建议用户简化查询或重试

6.1.3 自动重试机制

重试策略：

- 最大重试次数：3次
- 间隔时间：指数退避（2秒、4秒、8秒）
- 适用场景：相似度计算等关键步骤
- 效果：因临时网络抖动导致的失败率从15%降至3%

6.2 扩展性设计

6.2.1 水平扩展

当前架构：单机Docker Compose部署（1个Backend、1个Frontend、1个Neo4j、1个Redis）

扩展方案：

```
# 扩展Backend到3个副本
docker-compose up --scale backend=3 -d

# 添加Nginx负载均衡
nginx:
  image: nginx:alpine
  ports:  ["80:80"]
  volumes: ["/etc/nginx/nginx.conf"]
  depends_on: [backend]
```

6.2.2 Kubernetes就绪设计

预置K8s部署模板（infrastructure/k8s/）：

- **Deployment**：定义后端Pod副本数为3，配置资源限制（CPU 500m、内存1Gi）
- **Service**：后端ClusterIP类型，前端NodePort类型
- **Ingress**：域名路由规则（conceptgraph.example.com → 前端）
- **HPA**：自动扩缩容（CPU >70%增加副本，<30%缩减）

6.2.3 缓存层扩展

当前: Redis单机 (1小时TTL)

扩展方案一: Redis Cluster

- 3个主节点+3个从节点
- 数据分片和高可用

扩展方案二: 分层缓存

- L1: 本地缓存 (TTLCache, 5分钟)
- L2: Redis缓存 (1小时)

七、未来优化方向

1.性能优化

批量Embedding计算, 减少OpenAI API调用次数; **Redis集群** 支持高可用缓存

Neo4j索引优化: 加速图查询性能; **前端虚拟滚动**: 处理超大图谱(>100节点)

2.功能增强

WebSocket实时推送: 长时间LLM调用时推送进度; **用户反馈机制**: 收集概念关联质量评价

知识图谱导出: 支持GraphML/GEXF格式; **多语言支持**: 中英文切换

3.AI能力提升

多模型对比: 对比Gemini/GPT-4/Claude输出质量

知识库RAG: 集成向量数据库(Milvus/Qdrant)

域微调: 针对学术概念微调小模型

附录:

1.详细分工

成员A (肖璟仪 - ezzzb7) - 算法/Agent组

核心贡献:

1. Agent系统设计 (agents/)

- ConceptDiscoveryAgent: 概念挖掘Agent,实现CoT推理
- VerificationAgent: 知识校验Agent,多源验证逻辑
- GraphBuilderAgent: 图谱构建Agent
- AgentOrchestrator: 三层Agent编排器

2. Prompt工程 (prompts/)

- discovery_prompts.py: 跨学科挖掘Prompt(6大领域强制搜索)
- verification_prompts.py: 知识校验Prompt(3步验证法)
- graph_prompts.py: 图谱构建Prompt

3. 算法模块 (algorithms/)

- `semantic_similarity.py` : OpenAI Embeddings语义相似度计算
- `credibility_scorer.py` : 多源可信度评分算法
- `data_crawler.py` : Wikipedia + Arxiv数据爬虫
- `discipline_classifier.py` : 学科分类器

4. 真实LLM集成

- 集成OpenRouter + Gemini 3 Flash推理模式
- 实现语义相似度排序+动态可信度评分
- Wikipedia定义获取+学术概念过滤

5. 功能2和功能3实现

- 指定学科挖掘逻辑(`multi_function_generator.py`)
- 多概念桥梁发现算法(`generate_bridge_concepts`)

代表性提交:

- `feat(agent)`: 完成阶段一Agent编排系统 (d68a4a5)
- `feat`: 完成阶段二算法模块 (9264c17)
- `feat`: 完成阶段三知识校验层 (9a64e4f)
- `feat`: 实现真实LLM生成+语义相似度排序 (3dfc3dd)
- `feat`: 增加两项搜索功能 (3c4d2d2)

成员B (张凯诚 - kaifenger) - 架构/工程组

核心贡献:

1. 云原生基础设施

- Docker Compose编排(`docker-compose.yml`)
- Dockerfile(Backend + Frontend)
- 启动脚本(`start.bat`, `start.sh`, `start.ps1`)

2. 后端服务 (backend/)

- `main.py` : FastAPI主应用,生命周期管理
- `config.py` : 统一配置管理
- `api/routes.py` : 6个API接口实现(discover/expand/ai_chat等)
- `database/neo4j_client.py` : Neo4j图数据库客户端
- `database/redis_client.py` : Redis缓存客户端

3. 前端应用 (frontend/)

- React 18 + TypeScript 5项目搭建
- `GraphVisualization.tsx` : D3.js力导向图可视化
- `NodeDetailPanel.tsx` : 节点详情面板(8部分展示)
- `App.tsx` : 主应用逻辑

4. 三层缓存架构

- Neo4j持久化存储(第一优先级)
- Redis 1小时缓存(第二优先级)
- LLM实时生成(第三优先级)
- 批量保存方法 `save_graph_data()`

5. CI/CD Pipeline

- GitHub Actions自动化测试(.github/workflows/ci.yml)
 - 后端Python测试 + 前端TypeScript测试
 - Docker构建验证
- 代表性提交:**

- `feat(backend)`: 完成阶段二后端服务开发 (1f31d08)
- `feat(frontend)`: 完成阶段三前端开发 (194404)
- `feat(ci)`: 配置GitHub Actions CI/CD Pipeline (aaee244)
- 集成Neo4j持久化存储并优化数据流 (cbe1115)
- 优化缓存机制和修复简介截断问题 (b4148d6)
- `Fix Docker deployment` (d7c291d)

协作模式

阶段一(1月16日):

成员A: Agent框架搭建 + Prompt模板设计

成员B: Docker环境 + FastAPI项目骨架

阶段二(1月17日):

成员A: 算法模块开发(相似度计算、数据爬虫)

成员B: 后端API接口 + 数据库客户端

阶段三(1月17-18日):

成员A: 知识校验层 + 多源验证逻辑

成员B: 前端D3.js可视化

阶段四(1月18-21日):

成员A: 真实LLM集成 + 功能2/3开发

成员B: CI/CD Pipeline + Neo4j持久化

阶段五(1月21-22日):

成员A: 批量相似度优化 + 错误处理改进

成员B: 缓存优化 + Docker部署修复

2. 关键实现文件索引

Prompt工程

- `backend/api/real_node_generator.py` - 功能1跨学科概念生成核心逻辑
- `prompts/discovery_prompts.py` - CoT三步推理Prompt模板
- `prompts/multi_function_prompts.py` - 功能2/3 Prompt设计

智能体系统

- `agents/orchestrator.py` - Agent编排器
- `agents/concept_discovery_agent.py` - 概念挖掘Agent
- `agents/verification_agent.py` - 知识校验Agent
- `agents/llm_client.py` - LLM客户端 (OpenRouter + Gemini)

算法模块

- `algorithms/semantic_similarity.py` - 语义相似度计算 (OpenAI Embeddings)
- `algorithms/credibility_scorer.py` - 可信度评分算法
- `algorithms/data_crawler.py` - Wikipedia + Arxiv数据爬虫

后端核心

- `backend/main.py` - FastAPI主应用
- `backend/api/routes.py` - 6个核心API接口
- `backend/database/neo4j_client.py` - Neo4j图数据库客户端
- `backend/database/redis_client.py` - Redis缓存客户端

前端可视化

- `frontend/src/components/GraphVisualization.tsx` - D3.js力导向图
- `frontend/src/components/NodeDetailPanel.tsx` - 节点详情面板
- `frontend/src/App.tsx` - 主应用组件

Docker部署

- `docker-compose.yml` - 多服务编排配置
- `backend/Dockerfile` + `frontend/Dockerfile` - 容器镜像
- `start.bat` / `start.sh` - 一键启动脚本

注：所有文件路径相对于项目根目录，完整源代码见 [GitHub仓库](#)。